

A REPORT
ON
SYNTHETIC DATA GENERATION FOR NANOPORE SIGNALS

BY

Name(s) of the student
Rahul Gudia

ID No(s).
2024AAPS0028G

Discipline(s)
B. E. in Electronics and Commu-
nication

Acknowledgments

I would like to express my sincere gratitude to my project supervisor, Professor Verma, for their guidance, patience, and support throughout the duration of this project. His clear conceptual breakdown of nanopore physics, as well as the provision of the high-frequency open-pore and bead translocation data, were helpful in shaping the development of the synthetic simulator.

I also wish to thank the Center for Nanoscience and Engineering department for providing the resources to conduct this research.

Contents

1	Introduction	5
2	Literature review	7
3	Methodology	9
3.1	Application Design and Workflow	9
3.2	Synthetic Nanopore Signal Generation	10
3.3	Experimental-Data Import and Signal Preparation	12
3.4	Power-Spectrum Analysis and Error Calculation	14
3.5	Parameter Optimization and Spectral-Offset Correction	15
3.6	Visualization, Export and Input Validation	17
4	Results and discussions	19
4.1	Final Graphical User Interface	19
4.2	Synthetic and Experimental Time-Domain Signals	20
4.3	Power-Spectrum Matching and Optimization	22
5	Conclusions and future work	27
	Bibliography	29

Chapter 1

Introduction

Solid-state nanopores are nanoscale openings fabricated in thin insulating membranes. When a voltage is applied across a nanopore immersed in an electrolyte solution, ions move through the pore and produce a measurable ionic current. If a particle or biomolecule passes through the nanopore, it temporarily changes the ionic current by partially obstructing the flow of ions. These short-duration changes, commonly referred to as translocation or blockade events, can provide information about the particles moving through the pore.

Nanopore sensing has become an important technique for detecting and studying nanoscale particles and biological molecules. However, the accuracy of nanopore measurements depends not only on the detected events but also on the background fluctuations present in the recorded current. These fluctuations may arise from the nanopore membrane, electrolyte, electrodes, measurement electronics, environmental disturbances, and other physical processes. When the noise level is high, small particle events may become difficult to detect or distinguish from random variations in the signal.

The noise present in a nanopore recording is generally not described by a single constant value. Its behaviour may change with frequency and may contain several noise components. Low-frequency fluctuations, approximately frequency-independent noise, baseline drift, and measurement-system effects can all contribute to the final recorded signal. Therefore, nanopore signals must be studied in both the time domain and the frequency domain. The time-domain signal shows how the measured current changes with time, while frequency-domain analysis helps identify how the signal power is distributed across different frequencies.

Experimental nanopore recordings can also be difficult to analyse because they are often collected at high sampling frequencies. The bead-translocation data used in this project were recorded at approximately 200kHz, resulting in a large number of data points. Large datasets may require considerable memory and processing time, especially when complete recordings are loaded unnecessarily. A practical analysis system should therefore allow the user to select and process only the required portion of an experimental recording.

Synthetic signal generation provides a useful approach for studying nanopore measurements. A synthetic signal can be produced under controlled conditions, allowing different noise characteristics, event properties, and sampling conditions to be investigated independently. Synthetic signals may also be used to test event-detection methods, study the effects of noise, and evaluate signal-processing algorithms without requiring repeated experiments.

However, manually selecting synthetic parameters does not always produce a signal that accurately represents the experimental data. Two signals may appear similar in the time

domain while having significantly different frequency-domain characteristics. Visual comparison alone is therefore insufficient for evaluating the similarity between an experimental signal and a synthetic signal. A more objective approach is required to compare their spectral behaviour and automatically determine suitable synthetic parameters.

This project addresses this problem through the development of a MATLAB-based graphical user interface for synthetic signal generation and experimental spectrum matching. The application was initially developed using experimental bead-translocation data obtained from a solid-state nanopore system. Nevertheless, the final application was designed as a more general signal-processing tool so that it can also be applied to other experimental signals whose noise spectra need to be reproduced.

The application enables the user to generate synthetic signals containing configurable noise, baseline variation, and temporary signal events. It also allows experimental recordings to be imported in several commonly used data formats. Selected portions of the experimental data can then be compared with the generated signal in both the time and frequency domains.

A major feature of the project is the use of numerical optimization to improve the agreement between the experimental and synthetic power spectra. Instead of requiring the user to repeatedly change the synthetic parameters through trial and error, the application searches for a suitable combination of parameters that reduces the difference between the two spectra. The user can then inspect the resulting signals, view the optimized parameters, evaluate the remaining spectral error, and export the results.

The graphical interface was developed to provide a structured and user-friendly workflow. Separate sections of the application are used for synthetic signal generation, experimental-data loading, spectrum comparison, optimization, and result export. This reduces the need for users to edit MATLAB source code directly whenever a new dataset is analysed.

The project also investigates the presence of a possible constant offset between the experimental and synthetic spectra. This feature helps determine whether a remaining difference is mainly caused by an overall scaling factor or by a genuine mismatch in spectral shape. The application presents both the original and corrected comparisons so that the effect of the offset can be examined.

Chapter 2

Literature review

The principle of resistive-pulse sensing, originally pioneered through the Coulter counter, has evolved significantly with the advent of modern solid-state nanopores. These microscopic apertures allow researchers to detect and characterize individual particles, ranging from nanometer-scale DNA to 5 μm micro-beads, by monitoring transient ionic current blockades as the particles translocate through the pore [1]. However, accurately resolving these microscopic, microsecond-long events requires high-bandwidth transimpedance amplifiers.

As noted by [3], operating at high sampling frequencies (e.g., 200 kHz) inherently introduces significant measurement challenges, primarily in the form of thermal white noise and low-frequency $1/f$ flicker noise generated by both the recording electronics and the physical nanopore membrane itself. This extreme signal-to-noise degradation can easily obscure genuine particle translocations, making raw data extraction exceptionally difficult.

To mitigate these physical limitations, researchers increasingly rely on computational modeling and stochastic simulators to isolate nanopore physics [2]. Simulators provide a controlled mathematical environment where variables such as environmental baseline drift, stochastic particle dwell times (λ), and specific noise power spectrums can be precisely manipulated. By generating highly realistic synthetic signal traces, computational models enable engineers to rigorously test advanced signal-filtering algorithms and generate vast amounts of analytical training data, entirely bypassing the temporal and financial constraints of continuous physical laboratory trials.

Chapter 3

Methodology

This project followed a computational modelling and signal-comparison approach. A synthetic nanopore signal was first generated using user-defined parameters. Experimental nanopore data were then imported into the MATLAB application, and a selected portion of the experimental recording was prepared for comparison with the synthetic signal. The comparison was performed in the frequency domain by calculating and normalizing the power spectra of both signals. A numerical error value was used to represent the difference between the experimental and synthetic spectra. An optimization algorithm then adjusted selected synthetic parameters to minimize this error..

3.1 Application Design and Workflow

The project was developed as a MATLAB graphical user interface using MATLAB App Designer. The purpose of the interface was to combine synthetic signal generation, experimental-data loading, spectrum comparison, parameter optimization, visualization, and data export within a single application. The application was divided into four main tabs:

- Synthetic Signal
- Experimental Data
- Spectrum Matching
- Export

A common plotting area was used to display either the generated time-domain signal or the calculated frequency-domain spectra. The interface components, such as buttons, input fields, dropdown menus, and plotting axes, were defined as public application properties. Internally required variables, including the synthetic signal, experimental data, sampling frequencies, calculated spectra, optimized parameters, spectral error, and offset factor, were stored as private properties.

The general processing workflow was:

User input → Synthetic signal generation → Experimental data loading
→ Signal preparation → Power-spectrum calculation → Error calculation
→ Parameter optimization → Offset analysis → Visualization and export

This arrangement allowed data produced in one section of the application to be reused in another section. For example, the synthetic signal generated in the first tab was stored and later used during spectrum comparison and optimization.

The graphical interface reduced the need to repeatedly edit MATLAB code. The user could change the required parameters, load a new experimental recording, select a comparison segment, run the optimization, and export the results directly from the application.

3.2 Synthetic Nanopore Signal Generation

The synthetic signal was generated from a combination of background noise, baseline variation, and temporary blockade-like events. The user could control the signal using the following parameters:

Parameter	Description
F_s	Sampling frequency in hertz
T	Total signal duration in seconds
N_1	Magnitude of the frequency-independent noise component
N_2	Magnitude of the $1/f$ noise component
β	Parameter controlling the event or baseline behaviour
λ	Parameter controlling event duration or decay rate
N_{ev}	Number of generated events

The number of samples in the synthetic signal was determined from the sampling frequency and duration:

$$N = \text{round}(F_s T)$$

The time axis was constructed as:

$$t[n] = \frac{n}{F_s}, \quad n = 0, 1, \dots, N - 1$$

The interval between two consecutive samples was therefore:

$$\Delta t = \frac{1}{F_s}$$

The frequency vector required for noise generation was calculated as:

$$f[k] = \frac{k F_s}{N}$$

Because the first frequency value is zero, it would produce division by zero in a $1/f$ noise equation. Therefore, the zero-frequency value was temporarily replaced by a very small positive value during signal generation.

The frequency-dependent noise model was expressed as:

$$S(f) = N_1 + \frac{N_2}{f}$$

The term N_1 represented the approximately frequency-independent noise contribution, while $\frac{N_2}{f}$ represented the low-frequency $1/f$ -type contribution.

The amplitude of each frequency component was calculated from:

$$A(f) = \sqrt{S(f)}$$

Random phase values were then generated:

$$\phi_k = 2\pi r_k$$

where r_k was a random number between zero and one. The complex frequency-domain signal was formed as:

$$X[k] = A(f_k)e^{j\phi_k}$$

An inverse fast Fourier transform was applied to convert the frequency-domain representation into a time-domain noise signal:

$$x_{\text{noise}}[n] = \text{IFFT}\{X[k]\}$$

The mean value of the resulting noise was removed so that the noise fluctuated approximately around zero. Random phases were required because the noise-power equation alone only specifies the magnitude of each frequency component. Without random phase values, the inverse transform would produce a highly structured waveform instead of a random noise-like signal.

Temporary signal events were added to represent current changes that occur when particles enter or move through a nanopore. The number of events was defined by N_{ev} , which was rounded to the nearest integer.

Random event starting times were generated within the total signal duration. The event durations were determined using an exponential distribution:

$$\Delta_i = -\frac{\ln(r_i)}{\lambda}$$

Therefore, a larger value of λ generally generated shorter events, while a smaller value generated longer events.

During interactive signal generation, the events were represented as temporary negative pulses. These downward changes approximated current-blockade events in which a particle partially obstructs ionic flow through the nanopore.

A slowly varying baseline component was also included to represent gradual drift in the measured signal. In the interactive generator, this component was represented using a sinusoidal function such as:

$$d(t) = \sin(\beta t)$$

The complete synthetic signal was then formed by adding the noise, event, and baseline components:

$$x_{\text{syn}}(t) = x_{\text{noise}}(t) + x_{\text{events}}(t) + d(t)$$

The generated time vector, synthetic signal, and sampling frequency were stored in the application for later spectrum comparison and export.

3.3 Experimental-Data Import and Signal Preparation

The application was designed to import experimental recordings in several commonly used formats, including:

- binary files;
- CSV files;
- Microsoft Excel files; and
- Axon Binary Format files.

The user selected the file type and then selected the required recording using a file-selection dialog. Additional input fields allowed the user to specify the experimental sampling frequency, the starting time of the required segment, and a signal scale factor.

The scale factor was applied using:

$$x_{\text{scaled}} = C_{\text{scale}} x_{\text{raw}}$$

This was useful when the experimental data required unit conversion or numerical rescaling before comparison.

For large binary recordings, the complete file was not immediately loaded into memory. The binary data were assumed to be stored as single-precision values, with each value occupying four bytes. The approximate number of samples was determined from:

$$N_{\text{exp}} = \frac{\text{file size in bytes}}{4}$$

The total experimental duration was then calculated as:

$$T_{\text{exp}} = \frac{N_{\text{exp}}}{F_{s,\text{exp}}}$$

When the user selected a starting time, the corresponding starting sample was calculated as:

$$n_{\text{start}} = \lfloor t_{\text{start}} F_{s,\text{exp}} \rfloor + 1$$

The location of the required sample in the binary file was determined from:

$$B_{\text{start}} = (n_{\text{start}} - 1) \times 4$$

The file pointer was moved directly to this location, and only the required number of samples was read. This approach reduced memory use and processing time when working with large experimental recordings.

CSV and Excel data were imported using MATLAB's numerical data-reading functions. When the file contained two or more columns, the first column was treated as time and the second column as the measured signal.

The experimental sampling interval was estimated from the time values using:

$$\Delta t_{\text{exp}} = \text{median}(t_{n+1} - t_n)$$

The experimental sampling frequency was then calculated as:

$$F_{s,\text{exp}} = \frac{1}{\Delta t_{\text{exp}}}$$

The median time difference was used because it is less affected by isolated irregular values than a calculation based on only one pair of samples.

When the file contained only one column, it was assumed to contain signal values only. The experimental sampling frequency was then entered manually, and the time vector was reconstructed as:

$$t_n = \frac{n}{F_{s,\text{exp}}}$$

ABF recordings were imported using the external MATLAB function `abfload`. The function returned the sampling interval in microseconds, and the sampling frequency was calculated as:

$$F_{s,\text{exp}} = \frac{10^6}{s_i}$$

where s_i is the ABF sampling interval in microseconds.

Before spectrum comparison, a segment of the experimental data was selected with approximately the same duration as the synthetic signal. The experimental and synthetic time axes were shifted so that both began at zero:

$$t_{\text{exp,rel}} = t_{\text{exp}} - t_{\text{exp}}(1)$$

$$t_{\text{syn,rel}} = t_{\text{syn}} - t_{\text{syn}}(1)$$

The experimental signal was then interpolated onto the synthetic time positions:

$$x_{\text{exp,int}} = \text{interp1}(t_{\text{exp,rel}}, x_{\text{exp}}, t_{\text{syn,rel}})$$

Interpolation ensured that the experimental and synthetic signals had corresponding time positions. If the two arrays still had different lengths, both were truncated to the shorter length:

$$N_c = \min(N_{\text{exp}}, N_{\text{syn}})$$

This produced compatible signals for the later frequency-domain calculations.

3.4 Power-Spectrum Analysis and Error Calculation

The experimental and synthetic signals were compared in the frequency domain. The fast Fourier transform was calculated for each time-domain signal:

$$X[k] = \text{FFT}\{x[n]\}$$

The power spectrum was then calculated using:

$$S[k] = |X[k]|^2$$

Because the input signals were real-valued, the Fourier transform contained corresponding positive- and negative-frequency components. Only the non-negative-frequency half was retained:

$$K = \left\lfloor \frac{N}{2} \right\rfloor + 1$$

The frequency axis was calculated using:

$$f[k] = \frac{kF_s}{N}, \quad k = 0, 1, \dots, K - 1$$

The power spectrum was normalized to make the experimental and synthetic results more directly comparable. In the developed application, normalization was performed using the zero-frequency power:

$$S_n[k] = \frac{S[k]}{S[0]}$$

This made the normalized direct-current value equal to one:

$$S_n[0] = 1$$

If the zero-frequency power was invalid or equal to zero, the maximum spectrum value could be used as a fallback normalization value.

After normalization, the zero-frequency point was removed from the plotted spectrum because a log-log graph cannot display zero frequency. The logarithm of zero is undefined.

The experimental and synthetic spectra could contain different frequency positions. The comparison was therefore restricted to the common frequency range:

$$f_{\min} = \max[\min(f_{\text{exp}}), \min(f_{\text{syn}})]$$

$$f_{\max} = \min[\max(f_{\text{exp}}), \max(f_{\text{syn}})]$$

The synthetic spectrum was interpolated onto the experimental frequency positions within this common range. Invalid values, including NaN and infinite values, were excluded.

The difference between the spectra was quantified using a sum-of-squared-errors objective function:

$$\varepsilon = \sum_{k=1}^M [S_{n,\text{exp}}(f_k) - S_{n,\text{syn}}(f_k)]^2$$

A smaller value of ε indicated that the synthetic spectrum more closely followed the experimental spectrum. A larger value indicated a greater difference in spectral shape or magnitude.

This numerical error provided a more objective comparison than visually inspecting the two curves. It was also used as the objective function during parameter optimization.

3.5 Parameter Optimization and Spectral-Offset Correction

The purpose of the optimization process was to automatically determine synthetic parameters that reduced the difference between the experimental and synthetic power spectra.

The principal optimized parameters were:

$$\mathbf{p} = \begin{bmatrix} N_1 \\ N_2 \\ \beta \\ \lambda \end{bmatrix}$$

The number of events N_{ev} remained a user-defined integer rather than being directly optimized.

The optimization problem was expressed as:

$$\underset{N_1, N_2, \beta, \lambda}{\text{minimize}} \quad \varepsilon$$

Because all four parameters were required to remain positive and could have values differing by several orders of magnitude, the optimization was performed in logarithmic parameter space:

$$q_i = \log_{10}(p_i)$$

The actual parameters were recovered using:

$$p_i = 10^{q_i}$$

This transformation prevented the optimizer from producing negative values and improved the search across parameters with very different numerical scales.

Parameter limits were applied to prevent unrealistic or numerically unstable values. When a candidate parameter set was outside the selected limits, the objective function returned a very large penalty value. This discouraged the optimization algorithm from continuing in invalid regions.

For each candidate parameter set, the application generated a new synthetic signal, calculated its normalized spectrum, and evaluated the spectral error. A fixed random-number seed was used during this process. Fixing the seed ensured that the same random phases and event positions were used for every repeated evaluation.

Without a fixed seed, two evaluations using the same parameters could produce different error values only because a different random signal had been generated. This would make

the objective function unstable and could prevent the optimizer from determining whether a parameter change had genuinely improved the match.

MATLAB's `fminsearch` function was used to minimize the spectral error. This function uses a derivative-free Nelder–Mead search method and is suitable when an analytical derivative of the objective function is not available.

During optimization, the internal signal generator represented events as negative exponentially decaying functions:

$$e_i(t) = -\beta e^{-\lambda(t-t_i)}, \quad t \geq t_i$$

In this formulation, β controlled the initial event magnitude, while λ controlled how quickly the event returned toward the baseline.

The candidate synthetic signal was constructed as:

$$x_{\text{candidate}}(t) = x_{\text{noise}}(t) + \sum_{i=1}^{N_{ev}} e_i(t)$$

After the optimization was completed, the best parameters were converted back from logarithmic form and displayed in the corresponding input fields. The best synthetic spectrum, error value, and optimized parameters were stored for plotting and export.

An additional offset analysis was performed because the experimental and synthetic spectra sometimes showed an approximately constant vertical separation on a log–log graph. The logarithmic difference between the spectra was calculated as:

$$D_k = \log_{10}(S_{n,\text{syn}}(f_k)) - \log_{10}(S_{n,\text{exp}}(f_k))$$

The mean logarithmic difference was:

$$D_{\text{mean}} = \text{mean}(D_k)$$

The corresponding multiplicative offset factor was:

$$C_{\text{offset}} = 10^{D_{\text{mean}}}$$

The offset-corrected synthetic spectrum was calculated as:

$$S_{n,\text{syn,corr}} = \frac{S_{n,\text{syn}}}{C_{\text{offset}}}$$

The variation of the offset was also calculated using the standard deviation:

$$\sigma_{\text{offset}} = \text{std}(D_k)$$

A small value of σ_{offset} indicated that the separation was approximately constant across the analysed frequency range. A large value indicated that the difference was not caused only by a constant scale factor and that the spectral shapes were different.

The corrected spectrum was used mainly for interpretation and visualization. The optimization error itself was calculated from the raw normalized spectra.

3.6 Visualization, Export and Input Validation

The application displayed the results using both time-domain and frequency-domain plots. The time-domain plot showed the generated synthetic signal as a function of time. After comparison or optimization, the frequency-domain plot displayed:

- the experimental normalized spectrum;
- the raw synthetic normalized spectrum; and
- the offset-corrected synthetic spectrum.

The spectra could be displayed using linear axes or log-log axes. The log-log representation was particularly useful because frequency and power values could extend over several orders of magnitude. It also made low-frequency behaviour, $1/f$ -type trends, high-frequency differences, and constant multiplicative offsets easier to identify.

The plot title displayed the main comparison values, including the spectral error and the calculated offset factor. Legends, axis labels, and grid lines were included to improve interpretation.

The Export tab allowed the user to save the main results. The synthetic signal could be exported as a CSV file containing time and synthetic-signal values. The optimized parameter file contained the final values of N_1 , N_2 , β , λ , the number of events, the signal duration, the best error, and the offset factor.

The spectrum-data export contained:

- frequency in hertz;
- normalized experimental power;
- normalized synthetic power; and
- normalized offset-corrected synthetic power.

The displayed figures could also be exported in common image formats such as PNG, JPEG, and PDF.

Several validation procedures were included to reduce runtime errors. The application checked that:

- the sampling frequency was positive;
- the signal duration was positive;
- the number of events was an integer;
- a synthetic signal had been generated before comparison;
- experimental data had been loaded before comparison;
- the optimized parameters were positive before logarithmic transformation;
- a sampling frequency had been supplied for binary or single-column files;
- the selected binary file could be opened;

- `abfload` was available before importing ABF data;
- parameter values remained inside their permitted limits; and
- the calculated error was finite.

When an invalid operation or input was detected, the application displayed an alert explaining the problem. These validation procedures made the interface safer and easier to use, particularly for users who were unfamiliar with the internal MATLAB code.

Chapter 4

Results and discussions

The developed MATLAB application successfully combined synthetic nanopore signal generation, experimental-data loading, power-spectrum analysis, parameter optimization, offset correction, visualization, and data export within a single graphical interface. The performance of the application was evaluated using the generated time-domain signals and the comparison between the experimental and synthetic power spectra.

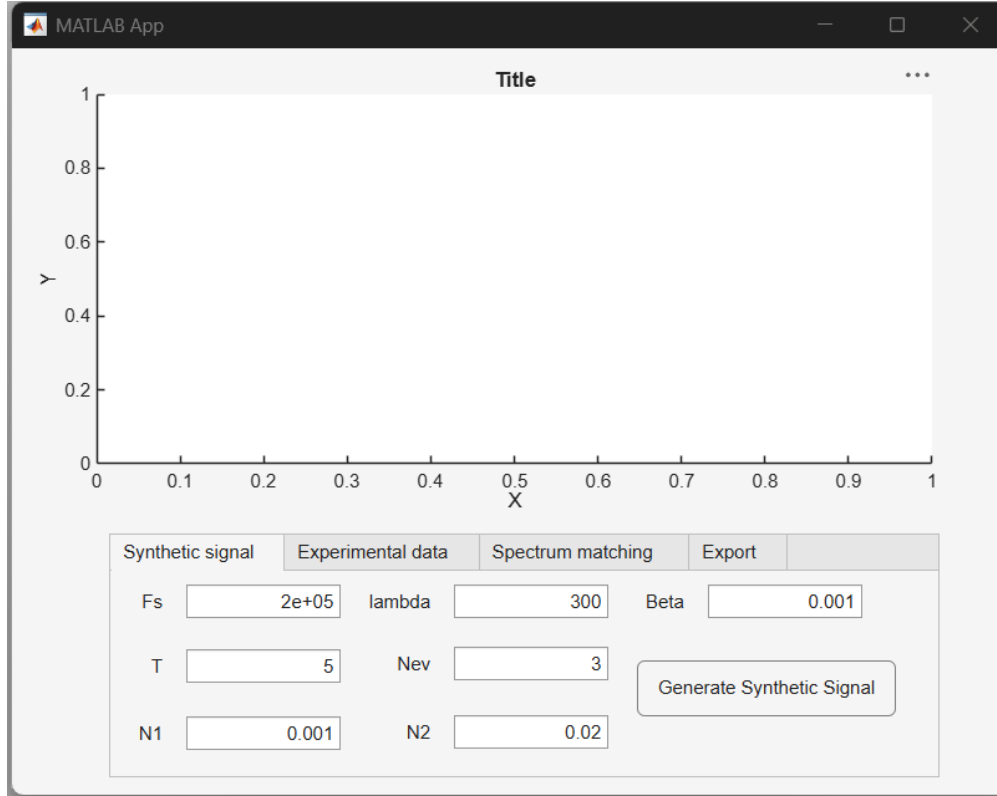
4.1 Final Graphical User Interface

The completed application was organised into four tabs: Synthetic Signal, Experimental Data, Spectrum Matching, and Export. This structure provided a clear sequence for carrying out the analysis without repeatedly editing the MATLAB code.

The Synthetic Signal tab allowed the user to enter parameters such as sampling frequency, signal duration, noise levels, event characteristics, and number of events. The Experimental Data tab was used to select the recording format, load the experimental file, specify the starting time of the required segment, and apply a scale factor where necessary.

The Spectrum Matching tab contained the controls for calculating the spectra, running the optimization, and displaying the error and offset factor. The Export tab allowed the final signals, spectra, parameters, and plots to be saved.

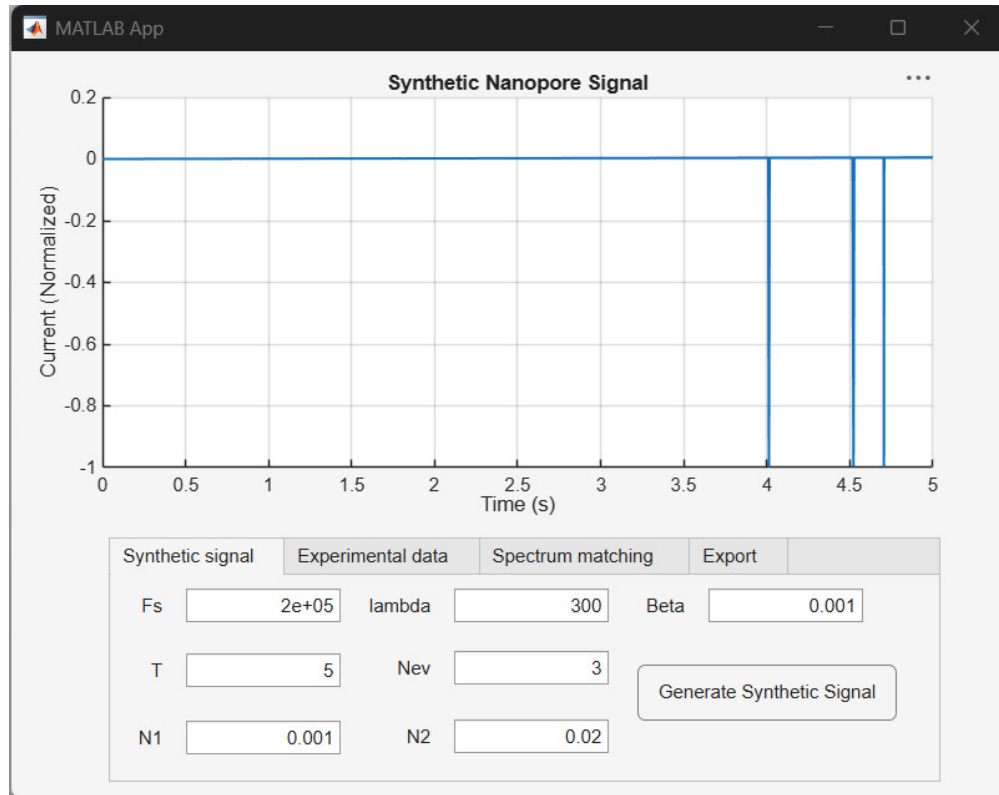
The interface simplified the complete analysis procedure and made the application easier to use. Input validation was also included so that the program displayed an alert when required data were missing or invalid values were entered.



MATLAB application developed for synthetic nanopore signal generation and experimental spectrum matching.

4.2 Synthetic and Experimental Time-Domain Signals

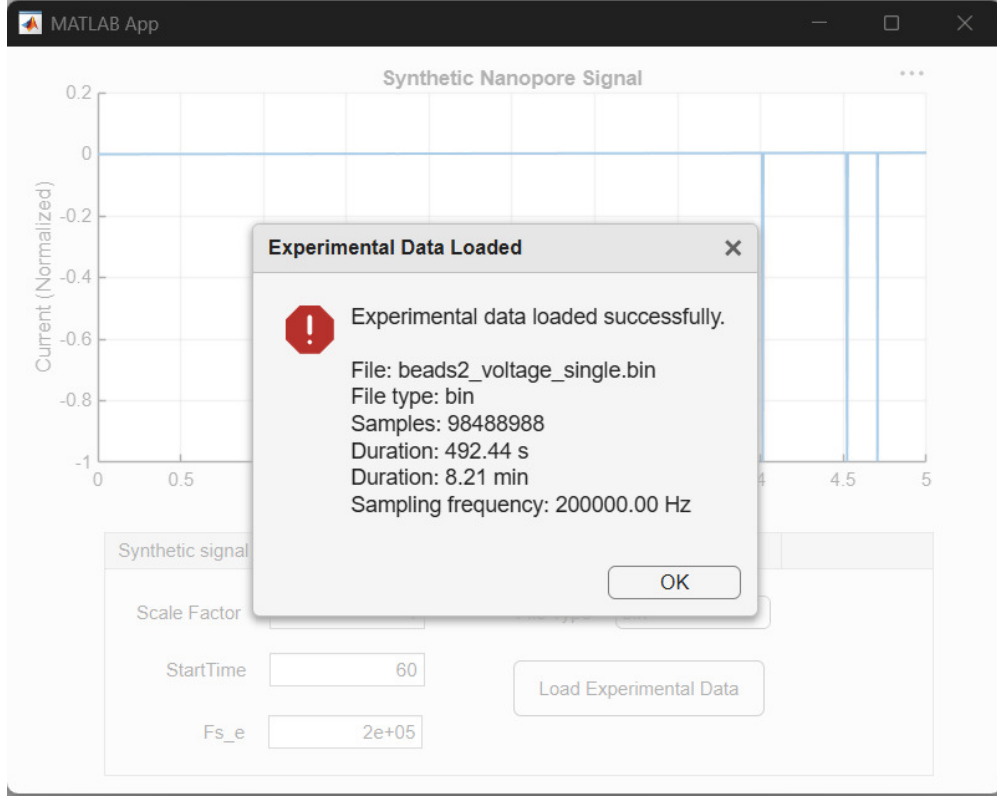
The synthetic signal was generated by combining frequency-dependent noise, temporary negative events, and baseline variation. The background fluctuations represented the noise present in a nanopore measurement, while the negative events represented temporary reductions in current caused by particles passing through or entering the pore.



Generated synthetic nanopore signal

The generated waveform showed random fluctuations rather than a repeating pattern because random phase values were used during noise generation. The number and duration of the events could be controlled through the application inputs.

The experimental bead recording was sampled at approximately 200kHz. Since the complete dataset contained a large number of samples, the application extracted only the segment required for comparison. This reduced memory usage and processing time. The app does not show a preview of the experimental data, only a confirmation that it has been loaded into the app.



Experimental nanopore recording used for comparison with the synthetic signal.

4.3 Power-Spectrum Matching and Optimization

The main comparison was performed using the normalized power spectra of the experimental and synthetic signals. The power spectra showed how the signal power was distributed over frequency and provided a more reliable comparison than examining only the time-domain waveforms.

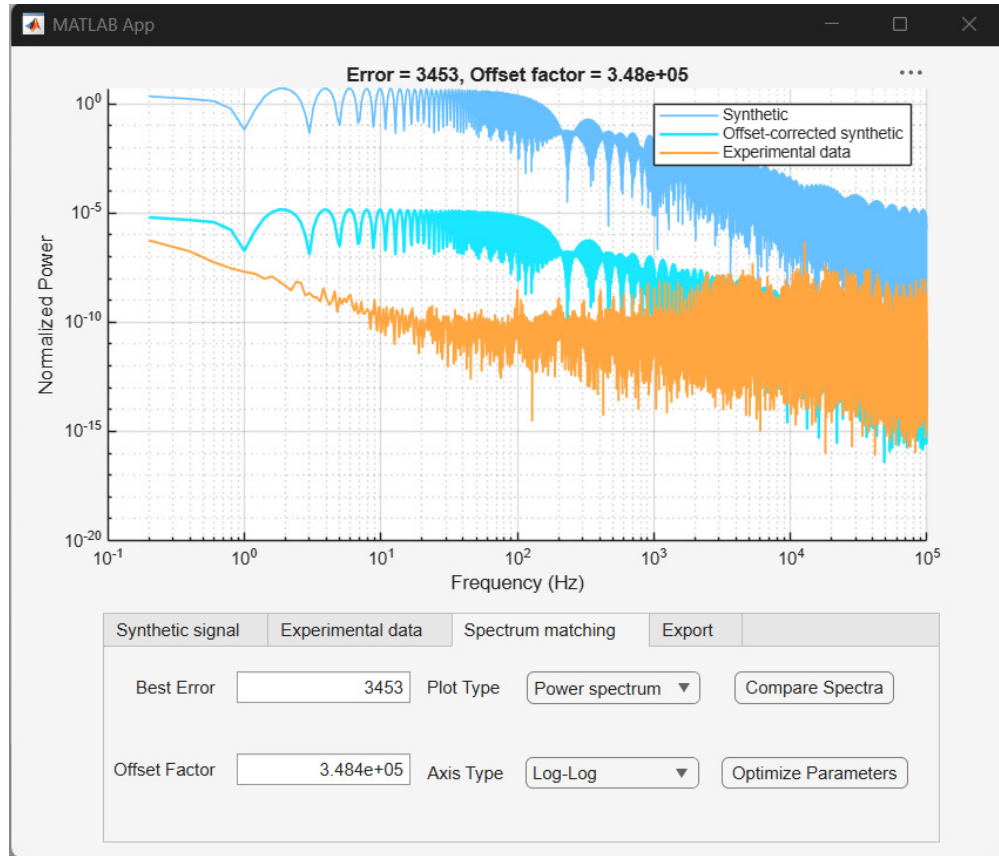
Before optimization, noticeable differences were present between the experimental and synthetic spectra. This indicated that the manually selected values of N_1 , N_2 , β , and λ did not completely reproduce the spectral behaviour of the experimental recording.

The difference between the two spectra was measured using the spectral error:

$$\varepsilon = \sum_k [S_{n,\text{exp}}(f_k) - S_{n,\text{syn}}(f_k)]^2$$

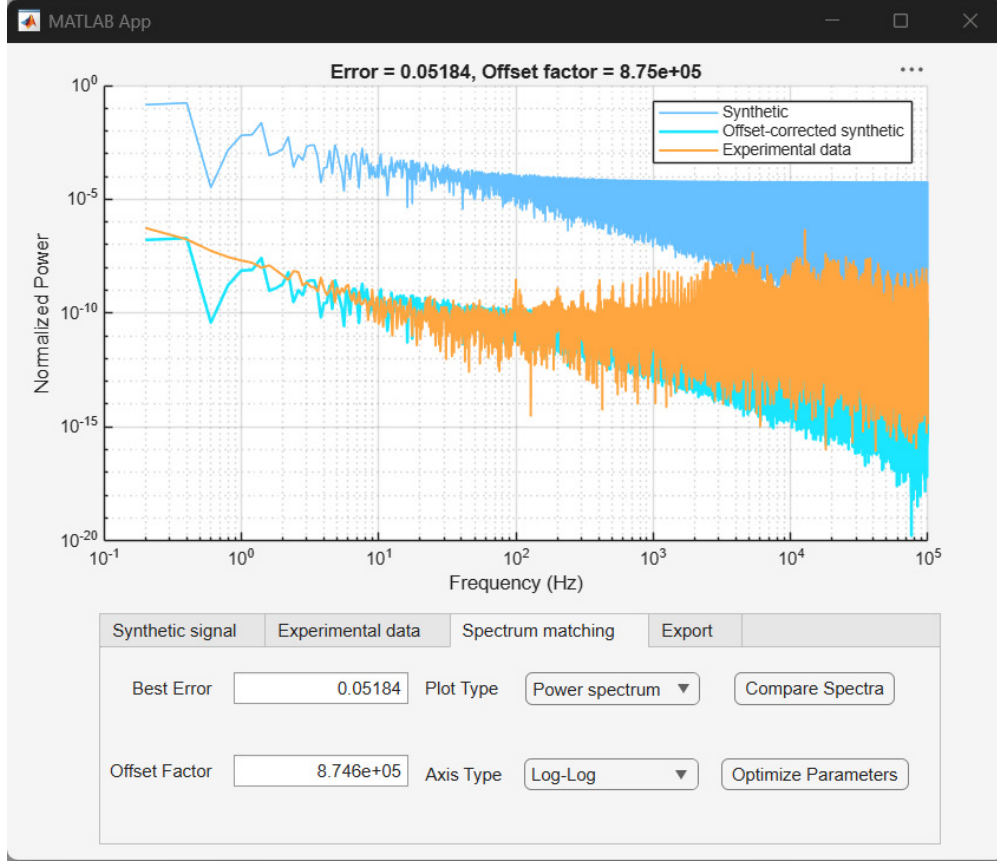
A smaller value of ε represented a closer spectral match.

MATLAB's optimization procedure adjusted the synthetic parameters to reduce this error. After optimization, the synthetic spectrum generally moved closer to the experimental spectrum over a significant part of the frequency range.



Normalized experimental and optimized synthetic power spectra.

An approximately constant vertical separation was also observed between portions of the experimental and synthetic spectra. The application calculated an offset factor and used it to shift the synthetic spectrum closer to the experimental spectrum.



Comparison of the experimental spectrum with the raw and offset-corrected synthetic spectra

The offset correction helped determine whether the difference was mainly caused by an overall scaling factor. However, it did not change the slope or shape of the synthetic spectrum. Therefore, any differences remaining after correction represented frequency-dependent behaviour that was not fully reproduced by the model.

The greatest mismatch was generally observed in the high-frequency region, where the synthetic spectrum appeared flatter or more sharply limited than the experimental spectrum. This may have resulted from filtering, amplifier response, capacitive effects, interpolation, or additional experimental noise sources that were not included in the simplified model:

$$S(f) = N_1 + \frac{N_2}{f}$$

Overall, the optimization improved the agreement between the spectra and provided a more systematic method than manually adjusting the parameters. Although the synthetic model did not reproduce every feature of the experimental recording, the application achieved its main objective of generating nanopore-like signals and adjusting their spectral behaviour to resemble measured data.

Table 4.1: Initial and optimized parameters

Parameter	Initial Value	Optimized Value
N_1	[0.001]	[3.772e-05]
N_2	[0.02]	[0.007815]
β	[0.001]	[0.06998]
λ	[300]	[1.001]
Spectral error	[3454]	[0.051834]

Chapter 5

Conclusions and future work

This project successfully developed a MATLAB-based graphical user interface for generating synthetic nanopore signals and comparing them with experimental recordings. The application combined synthetic signal generation, experimental-data loading, power-spectrum analysis, numerical optimization, spectral-offset correction, visualization, and data export within a single user-friendly system.

The synthetic signal was produced using configurable noise components and temporary blockade-like events. The application supported the analysis of experimental data stored in several formats and allowed selected segments of large recordings to be processed efficiently. Comparing the signals in the frequency domain provided a more reliable assessment than time-domain observation alone, since signals with similar waveforms may still have different spectral characteristics.

The optimization procedure adjusted the synthetic parameters to reduce the difference between the normalized experimental and synthetic power spectra. The results showed that optimization produced a closer spectral match than manual parameter selection. The offset analysis also helped determine whether part of the remaining difference was caused by a constant scaling factor or by a genuine difference in spectral shape.

Although the developed model reproduced the overall noise behaviour of the experimental signal, some differences remained, particularly in the high-frequency region. These differences may have resulted from filtering, amplifier response, capacitive effects, additional experimental noise sources, or the simplified form of the synthetic noise model. Therefore, the optimized parameters should be interpreted as model-fitting parameters rather than exact physical properties of the nanopore system.

Overall, the project achieved its main objective of creating a practical and reusable application for synthetic signal generation and optimization-based spectrum matching. The graphical interface reduced the need for direct code modification and provided a structured workflow suitable for students, researchers, and other users working with noisy experimental signals.

The project also provides a foundation for future improvements. Further development could include more advanced noise models, consistent event shapes throughout the application, additional optimization methods, automatic event detection, frequency-range selection, improved handling of large files, and support for more experimental data formats. As an open-source MATLAB application, the program can be modified and extended for nanopore research and other signal-processing applications.

Bibliography

- [1] Kaan Kececi and Ali Dinler. Review—recent applications of resistive-pulse sensing using 2d nanopores. *Journal of The Electrochemical Society*, 171(3):037505, mar 2024.
- [2] Yu Li, Renmin Han, Chongwei Bi, Mo Li, Sheng Wang, and Xin Gao. Deepsimulator: a deep simulator for nanopore sequencing. *Bioinformatics*, 34(17):2899–2908, 2018.
- [3] R. M. M. Smeets, U. F. Keyser, N. H. Dekker, and C. Dekker. Noise in solid-state nanopores. *Proceedings of the National Academy of Sciences*, 105(2):417–421, January 2008.